

Modelo Extra — AuraTracker 9000 (edición brainrot)

Programación Funcional — Lenguaje LISP

Alcance: repaso **rápido y liviano** de todo lo visto: predicado, clasificación con `cond`, MAPCAR, recursividad y función Menú. Las sublistas tienen **2 elementos** y los números son redondos, así que se resuelve en una clase sin sufrir. Es el mismo formato `(NOMBRE VALOR)` que la cátedra tomó en el segundo parcial de 2023 (el de las prendas y sus precios), pero con mejor ambientación.

LAS REGLAS DE SIEMPRE (AUNQUE EL TEMA SEA AL PEDO)

- Variables locales solo en el menú, nada de variables globales.
- No anidar IF con COND ni COND con IF.
- Validar los datos en el menú antes de procesar.
- Mensajes descriptivos al pedir el ingreso de datos.

El sitio **AuraTracker 9000** mide el aura de los personajes más famosos de internet. Cada personaje se registra en una sublista de **2 elementos**:

`(NOMBRE AURA)`

Donde:

- `NOMBRE` es un átomo con el nombre del personaje.
- `AURA` es un número entero: **positivo** si viene farmeando aura, **negativo** si la viene perdiendo (está *cocinado*).

1.- Lo primero que el sitio necesita es detectar a los que están mal. Desarrollar una **función predicado** que, a partir de los datos de **un** personaje, determine si está **cocinado**, es decir, si su aura es negativa. (1 p)

2.- En el perfil de cada personaje va un cartelito de estado. Definir una función que, a partir de los datos de **un** personaje, devuelva: (1,5 p)

- `(NOMBRE COOKED)` si su aura es negativa.
- `(NOMBRE MID)` si su aura está entre 0 y 1000 inclusive.
- `(NOMBRE SIGMA)` si su aura supera los 1000.

Hasta acá se trabajó con **un solo personaje**. El ranking completo es una lista de sublistas:

```
RANKING: ( (NOMBRE AURA) (NOMBRE AURA) (NOMBRE AURA) ... )
```

3.- Para la pantalla principal del sitio, desarrollar una función que, a partir del ranking, devuelva una nueva lista con el **nombre y el estado** de cada personaje. (1,5 p)

4.- Los del servidor quieren saber cuánta aura hay dando vueltas. Desarrollar una función que, a partir del ranking, calcule el **aura total acumulada** entre todos los personajes. (1,5 p)

5.- Para la sección *hall of shame*, desarrollar una función que, a partir del ranking, devuelva una nueva lista formada solamente por los **nombres de los personajes que están cocinados**. (2 p)

6.- Malas noticias: se aplicó el **fanum tax** y a todos les descuentan **100 de aura**. Desarrollar una función que, a partir del ranking, devuelva el **ranking actualizado**, es decir, una nueva lista donde cada personaje aparezca con su nombre y su aura ya descontada. (1,5 p)

7.- Desarrollar una función Menú que solicite al operador el ingreso del ranking, valide que sea una lista con datos, y muestre en consola el resultado de todas las funciones desarrolladas en los puntos anteriores. (1 p)

DESAFÍO (*puntos adicionales — 1 punto por ítem*)

a) SIX SEVEN. El sistema tiene que avisar cuando alguien queda con **exactamente 67** de aura, porque en ese caso hay que gritar *six seven*. Desarrollar una función que, a partir del ranking, devuelva la lista de **nombres** de los personajes cuya aura sea exactamente 67.

b) El más goated. Desarrollar una función que, a partir del ranking, devuelva el **nombre del personaje con más aura** de todos.

DATOS PARA PROBAR

```
(setq ranking '((TRALALERO 5000) (BOMBARDIRO -300) (TUNG-TUNG 67)
                (BALLERINA 1200) (CHIMPANZINI -50) (PATAPIM 900)))
```